Probabilistic Inference and State Estimation

INF8422: Robotic Perception and Spatial Intelligence

Prof. Pierre-Yves Lajoie

POLYTECHNIQUE MONTRÉAL

Agenda

1. Probabilistic Robotics
2. Probability Review
3. State Estimation: MLE & MAP
4. Factor Graphs
5. Nonlinear Optimization
6. Lie Groups
7. Summary – The Complete State Estimation Flow

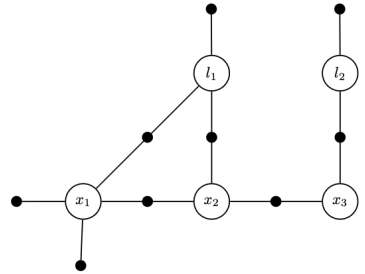
This Course

Last class, you saw SLAM in action:

- **Feature extraction, RANSAC, PnP** → geometric constraints
- **Bundle Adjustment** → reprojection error minimization
- **VIO factor graph** → optimization structure

Some terms were not well defined:

> *"Minimize the error"* – why least squares? What error in the probabilistic sense?
> *"Solve the graph"* – how? Which algorithm?



The SLAM factor graph – what we are going to formalize

This course answers these questions

Probability → Bayes → MLE/MAP → Least Squares → Factor Graphs → Gauss-Newton → Lie Groups

The foundations of SLAM back-end

Probabilistic Robotics

The measurement model

In probabilistic robotics, a measurement is a random variable.

$$z_t = h(x_t) + \epsilon_t$$

- z_t : The measurement vector at time t.
- x_t : The state of the robot/world (e.g., position, map).
- $h(\cdot)$: The **observation function** (physical model of the sensor).
- ϵ_t : The measurement noise (often Gaussian $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$).

Perception

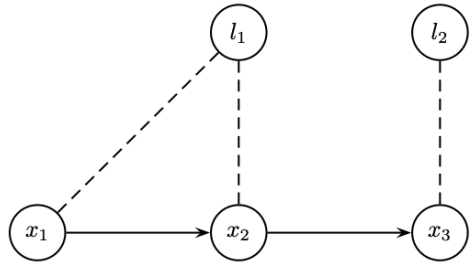
The goal of perception is to invert h to find x given z.

Uncertainty in Robotics

Unlike a perfect simulation, the real world is **uncertain**.

Sources of uncertainty:

- **Sensors**: Measurements are noisy. $z \neq h(x)$, i.e. $\|z - h(x)\|^2 > 0$.
- **Actuators**: Wheels slip, motors are not perfect. $u \neq \Delta x$.
- **Models**: Our equations are simplifications of the real physics.
- **Environment**: Objects move, lighting changes.



SLAM example: unknown poses and landmarks

Fundamental consequence

We can never know the robot's state with perfect precision. We must **quantify and propagate** this uncertainty.

Probabilistic Robotics

> *"Probabilistic robotics is a new approach to robotics that pays tribute to the uncertainty inherent in robot perception and action."* – Sebastian Thrun, 2005

Key idea:

Instead of saying *"The robot is at position* (x, y)*"*, we say:

"The robot's position follows a probability distribution $\mathcal{N}(\mu, \Sigma)$*"*

$$\Rightarrow \text{States} \longrightarrow \text{Probability distributions}$$

What this changes

- A **state** is a **probability density** $p(x)$.
- A **measurement** updates this density (Bayes' rule).
- An **action** propagates the uncertainty (error propagation).

In SLAM

We seek to estimate the conditional density:

$$P(X, M \mid Z)$$

- X: robot trajectory, M: map, Z: sensor measurements.

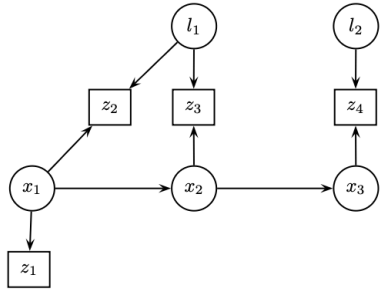
Probabilistic SLAM Modeling

Scenario: Trajectory with 3 poses x_1, x_2, x_3 and two landmarks l_1, l_2.

Joint density (via Bayesian network):

$$P(X, Z) = p(x_1)\, p(x_2|x_1)\, p(x_3|x_2)\, p(l_1)\, p(l_2)$$

$$\cdot\, p(z_1|x_1)\, p(z_2|x_1, l_1)\, p(z_3|x_2, l_1)\, p(z_4|x_3, l_2)$$

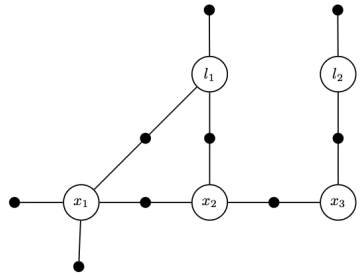


Bayesian network

Factor graphs

Better suited for inference $P(X|Z)$ than Bayesian networks. The conditional densities are replaced by **factors**:

$$P(X \mid Z) \propto \prod_i \phi_i(X_i)$$



Equivalent factor graph

Probability Review

Random Variables

Discrete case – PMF

A variable X takes values in $\{x_1, \ldots, x_n\}$.

$$P(X = x_i) = p_i, \quad \sum_{i=1}^{n} p_i = 1$$

Semantic classification

A camera sees an object. What is it?

- $P(X = \text{Bike}) = 0.1$
- $P(X = \text{Pedestrian}) = 0.8$
- $P(X = \text{Car}) = 0.1$

Continuous case – PDF

The density function $p(x)$ satisfies:

$$\int_{-\infty}^{\infty} p(x)\, dx = 1$$

Density ≠ Probability

- $P(X = x) = 0$.
- We compute: $P(a \leq X \leq b) = \int_a^b p(x)\, dx$.

Independence

If X and Y are independent:

$$p(x, y) = p(x) \cdot p(y)$$

Rules of Probability

Sum Rule (Marginalization):

$$p(x) = \int p(x, y)\, dy$$

Intuition: "flatten" the 2D distribution onto the x axis.

Product Rule:

$$p(x, y) = p(x \mid y)\, p(y) = p(y \mid x)\, p(x)$$

Chain Rule:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1})$$

Markov Assumption:

If x_{t+1} depends only on x_t (not on the history):

$$p(x_0, x_1, x_2) = p(x_0) \cdot p(x_1 \mid x_0) \cdot p(x_2 \mid x_1)$$

Conditional Independence

X and Y are independent given Z:

$$p(x, y \mid z) = p(x \mid z)\, p(y \mid z)$$

Bayes' Theorem in Robotics

$$\boxed{p(x \mid z) = \frac{p(z \mid x)\, p(x)}{p(z)}}$$

- $p(x)$: **Prior** – belief about the state before the measurement.
- $p(z \mid x)$: **Likelihood** – likelihood of the measurement given the state.
- $p(x \mid z)$: **Posterior** – belief about the state given the measurement.
- $p(z)$: **Evidence** – normalization.

Door sensor

State: door Open (O) or Closed (F). Prior: $P(O) = P(F) = 0.5$.

Noisy sensor: $P(z_O \mid O) = 0.8$, $P(z_O \mid F) = 0.4$.

If the sensor reports "Open":

$$P(z_O) = 0.8 \times 0.5 + 0.4 \times 0.5 = 0.6$$

$$P(O \mid z_O) = \frac{0.8 \times 0.5}{0.6} \approx \mathbf{0.67}$$

Even with an "Open" measurement, there is still a 33% chance that the door is closed!

The Normal (Gaussian) Distribution

The standard noise model in robotics – justified by the **Central Limit Theorem**.

Univariate

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- μ: Mean (center).
- σ^2: Variance (uncertainty).

68–95–99.7 rule:

- $\mu \pm 1\sigma$ → 68% probability.
- $\mu \pm 2\sigma$ → 95%.
- $\mu \pm 3\sigma$ → 99.7%.

Multivariate ($x \in \mathbb{R}^n$)

$$p(x) = \det(2\pi\Sigma)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)$$

- $\mu \in \mathbb{R}^n$: Mean vector.
- $\Sigma \in \mathbb{R}^{n\times n}$: Covariance matrix.

Mahalanobis Distance

$$D_M^2 = (x-\mu)^T\Sigma^{-1}(x-\mu)$$

Euclidean distance weighted by uncertainty.

Properties of Gaussians

Property 1 – Linear Transformation

Let $x \sim \mathcal{N}(\mu, \Sigma)$ and $y = Ax + b$.

$$\boxed{y \sim \mathcal{N}(A\mu + b,\ A\Sigma A^T)}$$

Example: change of frame camera → world. $A = R_C^W$.

$$\Sigma^W = R_C^W \, \Sigma^C \, R_W^C$$

Property 2 – Fusion (Product)

The product of two Gaussians is a Gaussian:

$$\mathcal{N}(\mu_{new}, \Sigma_{new}) \propto \mathcal{N}(\mu_1, \Sigma_1) \cdot \mathcal{N}(\mu_2, \Sigma_2)$$

The **information matrices** add up:

$$\boxed{\Sigma_{new}^{-1} = \Sigma_1^{-1} + \Sigma_2^{-1}}$$

Intuition

The more measurements we accumulate, the more the covariance Σ decreases. The uncertainty is determined by the data.

State Estimation: MLE & MAP

The Estimation Problem

Data:

- $Z = \{z_1, \ldots, z_m\}$: Noisy measurements.
- $u = \{u_1, \ldots, u_k\}$: Control commands (optional).

Unknown:

- x: The true state of the system (robot pose, map, calibration…).

Goal: Find the "best" estimate $\hat{x}$ that explains the data.

Measurement model:

$$z_i = h_i(x) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \Sigma_i)$$

The likelihood of a Gaussian measurement:

$$p(z_i \mid x) \propto \exp\left(-\frac{1}{2}\|h_i(x) - z_i\|^2_{\Sigma_i}\right)$$

Conditional independence:

$$p(z_{1:m} \mid x) = \prod_{i=1}^{m} p(z_i \mid x)$$

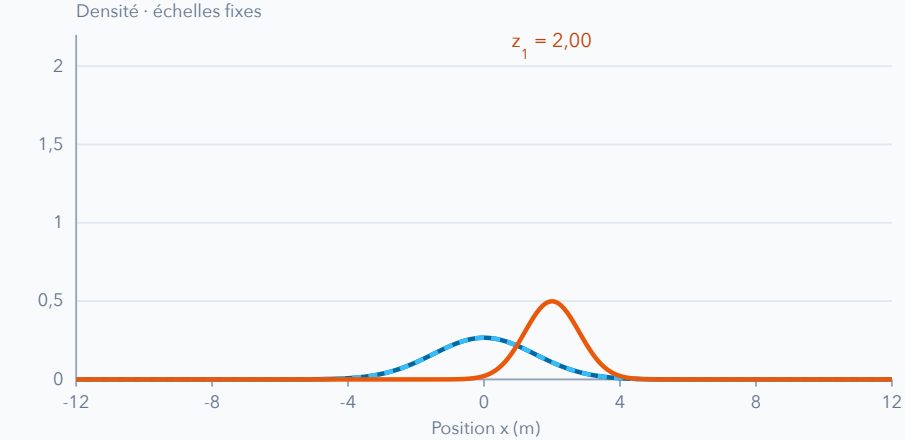
Two approaches

- **MLE** (Maximum Likelihood Estimation). - **MAP** (Maximum A Posteriori): incorporates a prior $p(x)$.

Bayesian update: one measurement at a time

Position **x fixe** · $z_k = x + \varepsilon_k$ · bruits gaussiens indépendants · σ = écart-type

1 Observer prior + mesure | **2 Fusionner** produit normalisé | **3 Retenir** posterior mémorisé



0 mesure intégrée $\sigma_0 = 1{,}50$

$$p_k(x) = \frac{p(z_k|x)\, p_{k-1}(x)}{\int p(z_k|u)\, p_{k-1}(u)\, du}$$

Avant z_1
μ = 0,00 · σ = 1,50

Mesure z_1
z = 2,00 · σ_z = 0,80

1. Une mesure à examiner

L'orange montre quels x rendent z_1 plausible. La mesure n'est pas encore intégrée.

Au départ, p_0 est aussi le prior de cette mise à jour : les deux courbes bleues se superposent.

$$\frac{1}{\sigma_k^2} = \frac{1}{\sigma_{k-1}^2} + \frac{1}{\sigma_z^2}$$

Les précisions s'additionnent.

μ_0 0,00 | σ_0 1,50 | z_1 2,00 | σ_z 0,80

Calculer le posterior → | ▶ Lecture | ↺ Réinitialiser

Régler la mesure, puis avancer.

Chaque cycle suppose une nouvelle observation indépendante, même si z est identique. Réutiliser la même observation compterait deux fois l'information.

MLE → Least Squares

Maximum Likelihood:

$$\hat{x}_{MLE} = \arg\max_x p(z_{1:m} \mid x) = \arg\max_x \prod_i p(z_i \mid x)$$

Log Trick (log is monotonically increasing):

$$= \arg\max_x \sum_i \ln p(z_i \mid x)$$

$$= \arg\max_x \sum_i \left(-\frac{1}{2} \|h_i(x) - z_i\|^2_{\Sigma_i} \right)$$

Maximizing the negative = Minimizing:

$$\boxed{\hat{x}_{MLE} = \arg\min_x \sum_i \frac{1}{2} \|h_i(x) - z_i\|^2_{\Sigma_i}}$$

Key takeaway

Under the Gaussian noise assumption, **Maximum Likelihood = Weighted Least Squares**.

$$\|e\|^2_\Sigma = e^T \Sigma^{-1} e$$

Precise sensor (σ small) → **large** weight $1/\sigma^2$.
Imprecise sensor (σ large) → **small** weight.

MAP → Least Squares with a Prior

Maximum A Posteriori:

$$\hat{x}_{MAP} = \arg\max_x \; p(x \mid Z) \propto p(Z \mid x)\, p(x)$$

If the prior is Gaussian: $x \sim \mathcal{N}(x_0, \Sigma_0)$, the problem becomes:

$$\boxed{\hat{x}_{MAP} = \arg\min_x \left[\sum_i \|h_i(x) - z_i\|^2_{\Sigma_i} + \|x - x_0\|^2_{\Sigma_0} \right]}$$

The prior acts as an **additional measurement** that pulls x toward x_0.

Example 1D – Odometry

Robot trajectory with 3 positions x_0, x_1, x_2. Displacement measurements: $d_1 = 2$ m, $d_2 = 1.5$ m ($\sigma_d = 0.5$).

Prior: $x_0 \approx 10$ m ($\sigma_0 = 0.1$).

$$E = \underbrace{\frac{(x_0 - 10)^2}{0.1^2}}_{\text{prior/anchoring}} + \underbrace{\frac{(x_1 - x_0 - 2)^2}{0.5^2}}_{\text{Motion 1}} + \underbrace{\frac{(x_2 - x_1 - 1.5)^2}{0.5^2}}_{\text{Motion 2}}$$

Solution: $\hat{x}_0 \approx 10, \hat{x}_1 \approx 12, \hat{x}_2 \approx 13.5$.

Without a prior: floating system

Without anchoring, if (x_0, x_1, x_2) is a solution, then $(x_0 + c, x_1 + c, x_2 + c)$ is one too. The system is underdetermined, and there are infinitely many solutions.

Intuition: The Spring Analogy

Each term of the cost function acts like a **spring** that pulls the state x toward the observed value z_i.

$$E(x) = \sum_i \underbrace{\frac{1}{2}\|h_i(x) - z_i\|^2_{\Sigma_i}}_{\text{energy of spring } i}$$

- Spring energy: $E = \frac{1}{2}k\Delta x^2$.
- Stiffness: $k = \Sigma_i^{-1}$ (inverse of the uncertainty/covariance).
- Elongation: $\Delta x = h(x) - z$ (measurement error).

$$\textbf{Minimize } E \iff$$

Find the mechanical equilibrium

Why the 1/σ² weights?

Not all measurements are born equal:

$$E(x) = \frac{(h_1(x) - z_1)^2}{\sigma_1^2} + \frac{(h_2(x) - z_2)^2}{\sigma_2^2} + \cdots$$

- **Precise** sensor (σ small) → **stiff** spring → **large** weight.
- **Imprecise** sensor (σ large) → **soft** spring → **small** weight.

The algorithm listens more to the precise sensors.

Springs: variance and loop closure

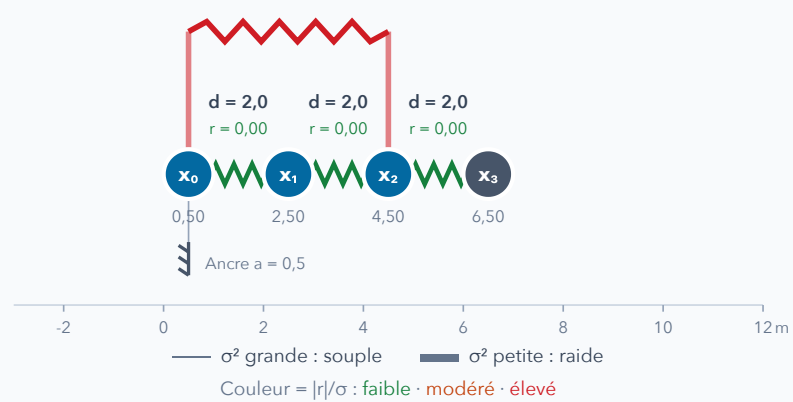
Chaîne | Boucle partielle : $x_0 \leftrightarrow x_2$

Positions en 1D · glisser les nœuds, puis relaxer

Boucle : $x_2 - x_0 \approx 3{,}0$ m

$r_{o2} = 1{,}00$ m

x_3 hors de la boucle



Coût J **5,000** Minimum J* **0,833** **À relaxer**

$$J(x) = \frac{1}{2}\sum_i \frac{r_i^2}{\sigma_i^2}, \qquad k_i = \frac{1}{\sigma_i^2}$$

Contrainte	Cible (m)	Variance σ² (m²)
Prior sur x_0 k = 10,0	0,5	0,10
$x_1 - x_0$ k = 4,0	2,0	0,25
$x_2 - x_1$ k = 4,0	2,0	0,25
$x_3 - x_2$ k = 4,0	2,0	0,25
Boucle $x_2 - x_0$ k = 10,0	3,0	0,10

Prior : $r_0 = x_0 - a$ · Lien : $r_{ij} = x_j - x_i - d_{ij}$

Conflit : le chemin 0→1→2 demande 4,0 m, la boucle 3,0 m. Les ressorts les moins précis absorbent davantage l'écart.
À l'équilibre, les forces se compensent ; les ressorts peuvent rester tendus. x_3 suit x_2 et conserve sa distance cible.

Relaxer → équilibre | II Pause | Perturber les positions | ↺ Réinitialiser

Animation : Newton amorti · modifier σ², puis relaxer

Robustness to Outliers – Weakness of Least Squares

Least squares minimizes $\sum_i \frac{1}{2}\|r_i\|^2$: the cost grows as r^2. A single outlier measurement (huge error) **dominates** the sum and **pulls** the whole solution.

Outlier measurements

Perceptual aliasing generates **false loop closures**. Fed as-is into an L_2 back-end, they **collapse** the map. The optimization must be made robust.

Robust kernels $\rho(r)$ – we replace $\frac{1}{2}r^2$ with a function that **caps**:

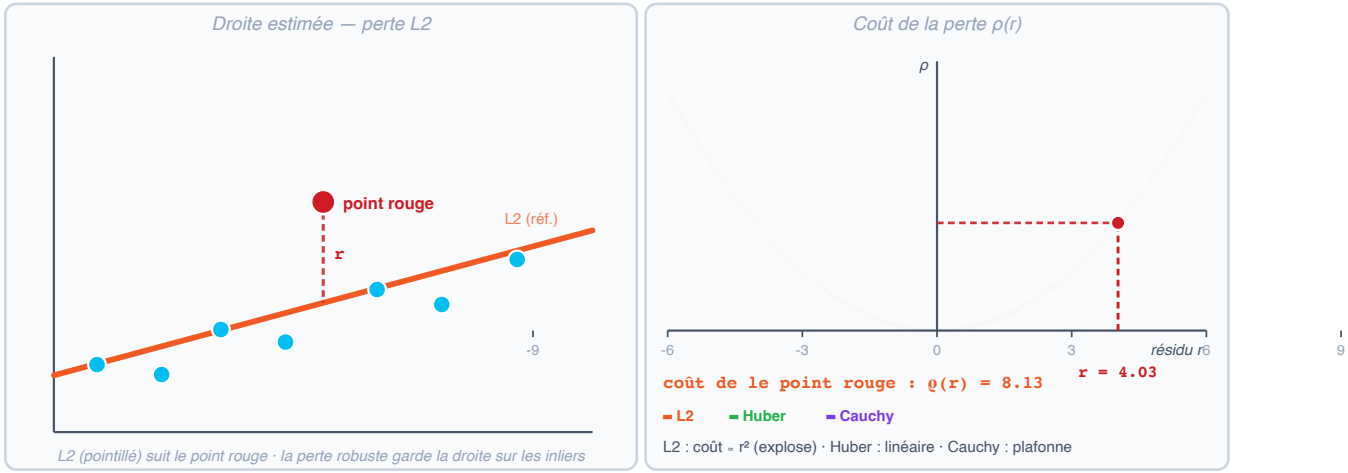
$$\hat{X} = \arg\min_X \sum_i \rho(\|r_i\|_{\Sigma_i})$$

- **Huber**: quadratic near 0, **linear** beyond δ.
- **Cauchy / Geman-McClure**: redescending – large residuals are **almost ignored**.

Advanced methods for robustness

Examples of advanced methods: *GNC* (Yang et al. 2020) gradually varies the robustness. *PCM* (Mangelson et al. 2018) geometrically filters inconsistent measurements.

Robust Estimation — Interactive Visualization



L2 | **Huber** | **Cauchy**

y du point rouge **9.2**

*Éloignez le point rouge : sous **L2** la droite le suit ; sous **Huber/Cauchy** son coût plafonne et la droite tient.*

Factor Graphs

From SLAM to Factor Graph

We have seen the factor graph for VIO:

$$\text{Poses } T_i \xrightarrow{[\text{IMU}]} T_{i+1}, \quad L_j \text{ seen from several } T_i$$

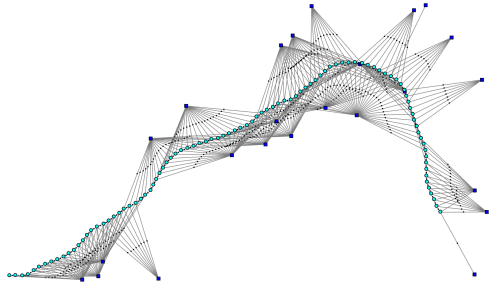
Now, let us state the **formal definition**.

A factor graph represents the **factorization of the joint probability** :

$$P(X \mid Z) \propto \prod_k \phi_k(X_k)$$

where each **factor** ϕ_k encodes a measurement (or a prior) as a Gaussian likelihood:

$$\phi_k(X_k) \propto \exp\left(-\frac{1}{2}\|h_k(X_k) - z_k\|^2_{\Sigma_k}\right)$$



SLAM factor graph

Connection to the previous course

Bundle Adjustment minimizes $\sum \|z_{ij} - h(T_j, P_i)\|^2$, which is exactly the MAP on this factor graph, with each reprojection as a factor.

Factor Graphs – Definition

A factor graph represents the **factorization of the joint probability** :

$$P(X \mid Z) \propto \prod_i \phi_i(X_i)$$

Elements :

- **Nodes (variables)** : Unknown states X_i (poses x_k, landmarks l_j, calibration…).
- **Factors** ϕ_i : Potential functions associated with the measurements Z and the priors.

The observed measurements are **incorporated as parameters** of the factors, not as nodes.

Advantages over Bayesian networks :

- Clear visual structure for inference.
- Directly linked to the sparse structure of the optimization problem.
- Easily implementable (GTSAM, g2o, Ceres).

Example: Odometry factor

$$\phi(x_i, x_{i+1}) \propto \exp\left(-\frac{1}{2}\|x_{i+1} \ominus x_i - \Delta u_i\|^2_\Sigma\right)$$

Connects two consecutive poses with the odometry measurement Δu_i.

MAP on Factor Graphs

MAP inference = maximize the product of all factors:

$$X^{MAP} = \arg\max_X \prod_i \phi_i(X_i)$$

With a Gaussian noise model for each factor:

$$\phi_i(X_i) \propto \exp\left(-\frac{1}{2}\|h_i(X_i) - z_i\|^2_{\Sigma_i}\right)$$

The **negative log** turns the problem into **non-linear least squares**:

$$\boxed{X^{MAP} = \arg\min_X \sum_i \|h_i(X_i) - z_i\|^2_{\Sigma_i}}$$

Natural sensor fusion

Minimizing the objective function $\sum_i \|h_i(X_i) - z_i\|^2_{\Sigma_i}$ automatically **combines** all measurement sources:

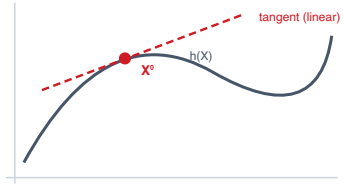
- Odometry (wheels, IMU)
- Visual observations (features, reprojection)
- LiDAR, GPS, loop closures…

Each factor contributes proportionally to its precision Σ_i^{-1}.

Why Linearize? The Problem is Non-Linear

MAP gave us a **least-squares** objective:

$$\hat{X} = \arg\min_{X} \sum_{i} \|h_i(X) - z_i\|^2_{\Sigma_i}$$



The h_i are often **non-linear** (projection, IMU…). In SLAM, we generally do not have a direct analytical solution. Gauss–Newton and LM solve a sequence of **linear systems**.

The strategy

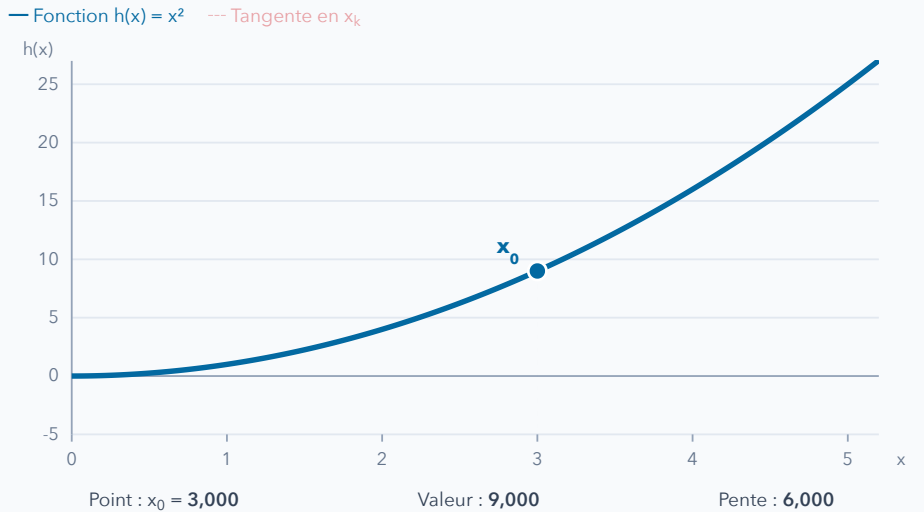
Linearize h around X^0 (1st-order Taylor), solve the **linearized least squares**, update X, then **iterate**.

Section outline (Lift–Solve–Retract)

1. **Lift**: linearize → Jacobian, whitening, matrix form.
2. **Solve**: normal equations + Cholesky (sparse).
3. **Retract + iterate**: Gauss-Newton / LM (next section).

Taylor Reminder: a local approximation

1 Choisir un point | 2 Tracer la tangente | 3 Voir l'erreur | 4 Calculer un pas



$$h(x_k + \Delta x) \approx h(x_k) + h'(x_k)\,\Delta x$$

1. Partir de ce qu'on connaît

On choisit un point x_k sur la courbe. Ici, **h(x) = x²** et **h'(x) = 2x**.

$h(x_k) = 9{,}00$
$h'(x_k) = 6{,}00$

La valeur et la pente à ce point vont définir notre approximation locale.

En plusieurs dimensions, la pente devient le Jacobien :

$$h(\mathbf{x} + \Delta\mathbf{x}) \approx h(\mathbf{x}) + J(\mathbf{x})\,\Delta\mathbf{x}$$

Point x_k **3,000** — Déplacement Δx **0,750**

Le point de développement reste fixe pendant le pas.

Étape suivante → | ▶ Démonstration | ↺ Réinitialiser

Taylor rend le prochain calcul plus simple. Il fournit une approximation locale ; la fonction d'origine reste non linéaire.

Linearization: the Jacobian

First-order Taylor expansion of h about X^0, for a perturbation Δ :

$$\boxed{h(X^0 + \Delta) \approx h(X^0) + J\,\Delta}$$

The Jacobian J

$$J = \left.\frac{\partial h}{\partial X}\right|_{X^0}, \qquad J_{ij} = \frac{\partial h_i}{\partial X_j}$$

Matrix of **partial derivatives** :

- **row** i = variation of residual i,
- **column** j = with respect to variable X_j.

1D SLAM

Odometry factor $r = (x_1 - x_0) - 2$. Its derivatives :

$$\frac{\partial r}{\partial x_0} = -1, \quad \frac{\partial r}{\partial x_1} = +1$$

→ Jacobian row $[-1,\ 1]$.

Here h is **linear** ⇒ J is **constant**.

We seek the correction Δ

At this iteration, $X^0 \in \mathbb{R}^n$ is **fixed**. We seek the displacement $\Delta = X - X^0$.

1. Change of variable

The initial problem is:

$$\hat{X} = \arg\min_X \sum_i \|h_i(X) - z_i\|^2_{\Sigma_i}.$$

Replacing X with $X^0 + \Delta$ gives:

$$\Delta^*_{\text{exact}} = \arg\min_\Delta \sum_i \|h_i(X^0 + \Delta) - z_i\|^2_{\Sigma_i}.$$

2. Linearize (local approximation)

At the fixed point X^0, Taylor gives:

$$\begin{aligned} h_i(X^0 + \Delta) - z_i \\ &\approx h_i(X^0) + J_i\Delta - z_i \\ &= J_i\Delta - \big(z_i - h_i(X^0)\big). \end{aligned}$$

We then minimize this **approximate model**:

$$\boxed{\Delta^* = \arg\min_\Delta \sum_i \big\|J_i\Delta - \big(z_i - h_i(X^0)\big)\big\|^2_{\Sigma_i}}$$

J_i and $z_i - h_i(X^0)$ are **constant** during this solve; only Δ varies.

Whitening (Noise whitening) ?

Each measurement can also have a *different uncertainty*. We cannot naively add them in a sum of squares.

The **Mahalanobis distance** weights each error by its uncertainty:

$$\|e\|_{\Sigma}^2 = e^\top \Sigma^{-1} e$$

A precise sensor (σ small) carries **more weight** (weight Σ^{-1}).

Whitening

We factor $\Sigma^{-1} = \Sigma^{-\top/2}\,\Sigma^{-1/2}$ and **change variables**:

$$e' = \Sigma^{-1/2}\, e \;\Rightarrow\; \|e'\|^2 = \|e\|_{\Sigma}^2$$

After whitening, all residuals have **unit** noise $\mathcal{N}(0, I)$, directly **comparable**.

$$A_i = \Sigma_i^{-1/2} J_i, \quad b_i = \Sigma_i^{-1/2}\big(z_i - h_i(X^0)\big)$$

From Least Squares to Matrix Form

$$A_i = \Sigma_i^{-1/2} J_i, \quad b_i = \Sigma_i^{-1/2}\big(z_i - h_i(X^0)\big)$$

We **stack** all the whitened factors. 1 row per residual:

$$A = \begin{bmatrix} A_1 \\ \vdots \\ A_m \end{bmatrix}, \qquad b = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}$$

The problem becomes a **linear least squares**:

$$\boxed{\Delta^* = \arg\min_{\Delta} \|A\Delta - b\|^2}$$

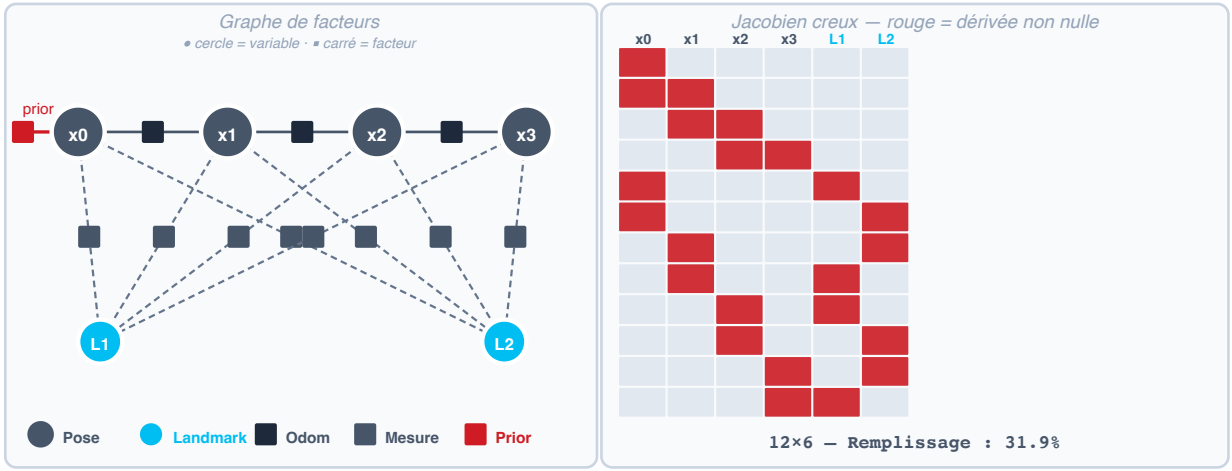
- **1 row** = 1 factor · **1 column** = 1 degree of freedom.
- Each row touches only **a few columns** ⇒ A is said to be **sparse**.

Our 1D SLAM with a loop (4 poses)

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ -1 & 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 2 \\ 2 \\ 1 \\ 5 \end{bmatrix}$$

5 factors: anchor x_0, 3 odometry measurements, 1 **loop closure** $x_3 - x_0$ (last row).

Factor Graph and Sparse Jacobian



Poses 4 Landmarks 2

12 facteurs × 6 variables – Le graphe grandit mais la Jacobienne reste creuse !

Derivation of the Normal Equations

We minimize $F(\Delta) = \frac{1}{2}\|A\Delta - b\|^2$
$= \frac{1}{2}(A\Delta - b)^\top (A\Delta - b)$.

Gradient (matrix differentiation):

$$\nabla_\Delta F = A^\top (A\Delta - b)$$

Optimality condition $\nabla_\Delta F = 0$:

$$\boxed{A^\top A\, \Delta^* = A^\top b}$$

Properties of $\Lambda = A^\top A$

- **Symmetric**: $\Lambda^\top = \Lambda$.
- **Positive definite** (hence invertible) if A has **full column rank**, a well-constrained problem.

Our 1D SLAM

$$\Lambda = \begin{bmatrix} 3 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix}, \; A^\top b = \begin{bmatrix} -7 \\ 0 \\ 1 \\ 6 \end{bmatrix}$$

Reminder: Solving a Linear System

Problems where we seek Δ such that $W\Delta \approx v$. Two cases:

- W **square and invertible**: unique solution $\Delta = W^{-1}v$, but we **never invert** explicitly (costly and unstable).
- W **rectangular**, more measurements than unknowns ($m > n$): **overdetermined** system ⇒ no unique solution ⇒ **least squares**.

Families of methods

- **Direct**: Gaussian elimination, **Cholesky**, QR – finite number of operations, exact solution (up to rounding).
- **Iterative**: conjugate gradient (CG)… for very large systems.

This course: **sparse Cholesky** on the normal equations.

Never invert

Computing Λ^{-1} is $O(n^3)$ **and destroys sparsity**.

Instead, we **factorize** then **substitute**.

Elimination, Ordering and Fill-in

Normal equations:

$$\Lambda = A^\top A$$

$$\boxed{\Lambda \Delta^* = A^\top b}$$

Factoring Λ = **eliminating** the variables one by one (Gaussian elimination / Cholesky).

Eliminating a variable **connects all of its neighbors to one another** in the graph → **new** edges = **new** non-zeros in Λ: the **fill-in**.

More fill-in ⇒ Λ less sparse ⇒ more costly factorization.

Ordering matters enormously

The **elimination ordering** determines the fill-in:

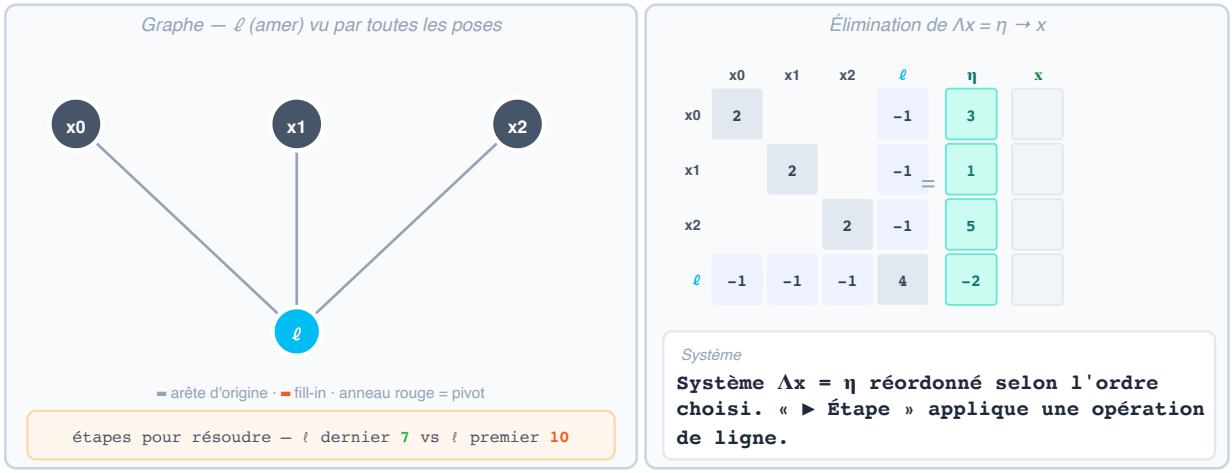
- landmark seen by all poses, eliminated **last** → **0 fill-in**.
- the **same** one eliminated **first** → links all poses → **massive** fill-in.

In practice

Heuristics (**COLAMD**, **AMD**) compute an ordering that **reduces the fill-in** before factoring – a key ingredient of SLAM solvers (GTSAM, Ceres, g2o).

Ordering and Fill-in – Visualization

We **solve** $\Lambda x = \eta$ by elimination in two orders: (ℓ last) vs (ℓ first).



Cholesky Factorization

Since Λ is **symmetric positive definite**, it admits a **triangular square root** :

$$\boxed{\Lambda = M^\top M}$$

M **upper triangular** (unique). It is the matrix analogue of $\lambda = (\sqrt{\lambda})^2$.

Why triangular? A triangular system is solved by **substitution** in $O(n^2)$ and much less if sparse.

Solving in 2 steps

To solve $\Lambda\,\Delta = M^\top M\,\Delta = A^\top b$:

1. **Forward substitution** : $M^\top y = A^\top b$ (top to bottom)
2. **Back substitution** : $M\,\Delta = y$ (bottom to top)

Sparsity preserved

With a good ordering, M stays **sparse** → near-linear factorization. This is what makes real-time SLAM possible.

Cholesky Factorization

We seek M **upper triangular**, with a positive diagonal, such that $\Lambda = M^\top M$.

$$\underbrace{\begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}}_{\Lambda} = \underbrace{\begin{bmatrix} a & 0 \\ b & c \end{bmatrix}}_{M^\top} \underbrace{\begin{bmatrix} a & b \\ 0 & c \end{bmatrix}}_{M} = \begin{bmatrix} a^2 & ab \\ ab & b^2 + c^2 \end{bmatrix}$$

- **Entry (1,1)** : $a^2 = 4 \quad \Rightarrow \quad a = \sqrt{4} = 2.$
- **Entry (1,2)** : $ab = 2 \quad \Rightarrow \quad b = 2/a = 1.$
- **Entry (2,2)** : $b^2 + c^2 = 5 \quad \Rightarrow \quad c = \sqrt{5 - b^2} = 2.$

$$\boxed{M = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}}$$

We identify the coefficients, not just an entry-by-entry square root.

Existence and uniqueness with this convention if Λ is symmetric positive definite.

For $\Lambda = A^\top A$, this requires A to have full column rank (in SLAM, this implies having a prior).

Cholesky

Entry (j, i) of $M^\top M$ is the **dot product of columns j and i** of M.

$$\Lambda_{ji} = \sum_{k=1}^{j} M_{kj} M_{ki} = \underbrace{\sum_{k<j} M_{kj} M_{ki}}_{\text{previous rows: known}} + M_{jj} M_{ji} \qquad (i \geq j)$$

1. Start with the diagonal

$$\Lambda_{jj} = \sum_{k<j} M_{kj}^2 + M_{jj}^2$$

$$\boxed{M_{jj} = \sqrt{\Lambda_{jj} - \sum_{k<j} M_{kj}^2}}$$

We subtract the squares already known, then choose the **positive root**.

2. Fill in the row to the right

$$\Lambda_{ji} = \sum_{k<j} M_{kj} M_{ki} + M_{jj} M_{ji}$$

$$\boxed{M_{ji} = \frac{\Lambda_{ji} - \sum_{k<j} M_{kj} M_{ki}}{M_{jj}}} \quad (i > j)$$

We subtract the products already known, then **divide by the pivot M_{jj}**.

We repeat for $j = 1, \ldots, n$. On the first row, the sums are empty, hence zero.

Why this order? Each computation uses only the previous rows and the pivot already computed.

Cholesky step by step: building M

On our 1D SLAM example: **write the equality → isolate the unknown → compute**.

Ligne 1 de M · coefficient 1 / 10 — **Diagonale → racine positive**

Λ donnée, fixe

3	−1	0	−1
−1	2	−1	0
0	−1	2	−1
−1	0	−1	2

$\mathbf{M}$ à construire

?	·	·	·
0	·	·	·
0	0	·	·
0	0	0	·

Donnée utilisée | Valeurs réutilisées | **Inconnue**

Les zéros sous la diagonale sont imposés.
On remplit **une ligne à la fois**, de gauche à droite.

Chaque entrée de Λ est un **produit scalaire de deux colonnes de M**.

1 · Écrire l'entrée (1, 1) du produit $M^T M$

$$\Lambda_{11} = \underbrace{M_{11}^2}_{\text{à déterminer}}$$

2 · Isoler la seule inconnue

$$M_{11} = \sqrt{\Lambda_{11}}$$

Quelle valeur doit remplacer « ? » pour que l'égalité soit vraie ?
Cliquez sur « Calculer » pour la révéler.

← Précédent | Calculer | ↺ Recommencer

Affichage arrondi à 3 décimales ; calculs non arrondis.

M is built: solving becomes simple

Solving in 2 steps

To solve $\Lambda\,\Delta = M^\top M\,\Delta = A^\top b$:

1. **Forward substitution** : $M^\top y = A^\top b$ (top to bottom)
2. **Backward substitution** : $M\,\Delta = y$ (bottom to top)

1. Forward substitution : $M^\top y = A^\top b$

$$\begin{bmatrix} 1.732 & 0 & 0 & 0 \\ -0.577 & 1.291 & 0 & 0 \\ 0 & -0.775 & 1.183 & 0 \\ -0.577 & -0.258 & -1.014 & 0.756 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} \approx \begin{bmatrix} -7 \\ 0 \\ 1 \\ 6 \end{bmatrix}$$

$$y \approx (-4.041,\ -1.807,\ -0.338,\ 3.780)^\top$$

We compute y_1, then y_2, y_3 and y_4.

No matrix to invert.

2. Backward substitution : $M\Delta = y$

$$\begin{bmatrix} 1.732 & -0.577 & 0 & -0.577 \\ 0 & 1.291 & -0.775 & -0.258 \\ 0 & 0 & 1.183 & -1.014 \\ 0 & 0 & 0 & 0.756 \end{bmatrix} \begin{bmatrix} \Delta_1 \\ \Delta_2 \\ \Delta_3 \\ \Delta_4 \end{bmatrix} \approx \begin{bmatrix} -4.041 \\ -1.807 \\ -0.338 \\ 3.780 \end{bmatrix}$$

$$\boxed{\Delta = (0,\ 2,\ 4,\ 5)^\top}$$

We compute Δ_4, then Δ_3, Δ_2 and Δ_1.

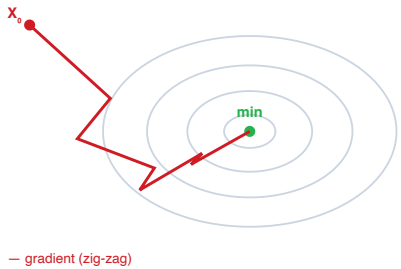
Nonlinear Optimization

Recap: Optimizing Means Descending Toward the Minimum

Minimize $F(X)$ **iteratively**: start from X_k, choose a **descent direction** d (one that decreases F), take a step α:

$$X_{k+1} = X_k + \alpha\,\Delta$$

- **Gradient descent**: $\Delta = -\nabla F$ (steepest slope). Simple but **slow**.
- **Newton**: uses the **curvature** (Hessian H), $\Delta = -H^{-1}\nabla F$. **Fast** near the minimum.



Gradient, Jacobian and Hessian – Least Squares

Cost: $F(X) = \frac{1}{2}\|r(X)\|^2$, residual $r(X) = h(X) - z$.

Jacobian of the residual: $J = \dfrac{\partial r}{\partial X}$.

Gradient (chain rule):

$$\nabla F = J^\top r$$

Exact **Hessian** $= J^\top J + \sum_i r_i \, \nabla^2 r_i$.

The Gauss-Newton algorithm, which we will see, **neglects** the 2nd term and approximates:

$$H \approx J^\top J$$

The bridge with the previous section

The Newton step $H\,\Delta = -\nabla F$ becomes:

$$\boxed{J^\top J\,\Delta = -J^\top r}$$

These are **exactly** the normal equations $A^\top A\,\Delta = A^\top b$ with

$$A = J \text{ (whitened Jacobian)}, \quad b = -r$$

$H \approx J^\top J$ requires **only the Jacobian** (1st derivatives) – no second derivatives.

Gauss-Newton Algorithm

We chain **linearize → solve → update**, in a loop:

1. **Linearize**: $r(X_k + \Delta) \approx r(X_k) + J_k\,\Delta$.
2. **Solve** the normal equations (Cholesky):

$$(J_k^\top J_k)\,\Delta^* = -J_k^\top\, r(X_k)$$

3. **Update**: $X_{k+1} = X_k \oplus \Delta^*$.

Repeat until $\|\Delta^*\| < \epsilon$.

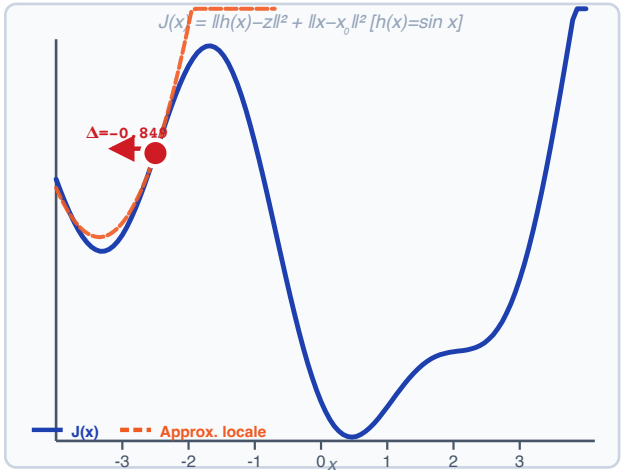
Pseudocode:

```
X ← X₀                              # initial linearization point
Repeat:
  r ← h(X) - z                      # raw residual
  J ← ∂h/∂X |_X                     # Jacobian
  r, J ← W·r, W·J                   # whitening: W = Σ^(-1/2)
  g ← Jᵀ·r                          # gradient ∇F   (-Jᵀr)
  H ← Jᵀ·J                          # approx. Hessian
  Δ ← solve H·Δ = -g                # Cholesky: H = MᵀM
  X ← X ⊕ Δ                         # composition (≠ addition)
Until ‖Δ‖ < ε   (or ‖g‖ < ε,  i > i_max)
```

Convergence

Quadratic near the minimum (very fast), but may **diverge** far away if the step is too large.

Gauss-Newton Convergence – Visualization



État de l'optimisation

Itération k = 0

`x_k = −2.5000  J = 1.8316`

Pas Gauss-Newton

`r = [h(x)−z, prior] = [-1.098, -0.791]`

`J = [∂h/∂x, ...] = [-0.801, 0.316]`

`Δ* = −(JᵀJ)⁻¹Jᵀr = −0.8493`

En cours...

|Δ| = 0.8493

`Historique x_k :`

`−2.500`

x_0 **−2.5** k = 0 / 12 **▶ 1 pas** **▶▶ Tout exécuter** **↺ Reset** *Convergence quadratique près du min, peut diverger loin*

Levenberg-Marquardt: Robustness

The Gauss-Newton problem: far from the minimum, the step Δ can be too large and diverge.

Solution: damping λ on the diagonal of H:

$$\boxed{(J^T J + \lambda \, \mathrm{diag}(J^T J)) \, \Delta = -J^T r}$$

- $\lambda \to 0$**:** recovers the Gauss-Newton step.
- **large λ:** small preconditioned gradient step (see next slide).

λ is adjusted **dynamically** at each iteration according to a gain ratio ρ.

Trust Region strategy:

Compute $\rho = \frac{\text{Actual gain}}{\text{Predicted gain}}$.

- $\rho > 0$ accept (cost decreases), decrease λ (→ GN).
- $\rho \leq 0$ reject (cost increases), increase λ (→ preconditioned gradient).

Algorithm	Speed	Robustness
Gradient Descent	Slow	High
Gauss-Newton	Very fast	Low
Levenberg-Marquardt	**Fast**	**High**

LM: Gauss–Newton, gradient… or in between?

At one iteration: $H = J^\top J$, $g = J^\top r = \nabla F$ and $D = \mathrm{diag}(H)$.

$$\boxed{(H + \lambda D)\,\Delta = -g} \quad \Longleftrightarrow \quad \boxed{\Delta_{\mathrm{LM}} = -(H + \lambda D)^{-1} g}$$

1. No damping: $\lambda = 0$

$$H\Delta = -g$$

$$\boxed{\Delta = -H^{-1} g}$$

We recover **exactly Gauss–Newton**.

For small λ, the step is close to the GN step.

2. Strong damping: λD dominates H

$$\lambda D \Delta \approx -g$$

$$\boxed{\Delta \approx -\frac{1}{\lambda} D^{-1} g}$$

Small **preconditioned gradient** step: D^{-1} adjusts the scale of the coordinates.

With $D = I$: $\Delta \approx -\alpha \nabla F$, where $\alpha = 1/\lambda$.

3. Intermediate damping

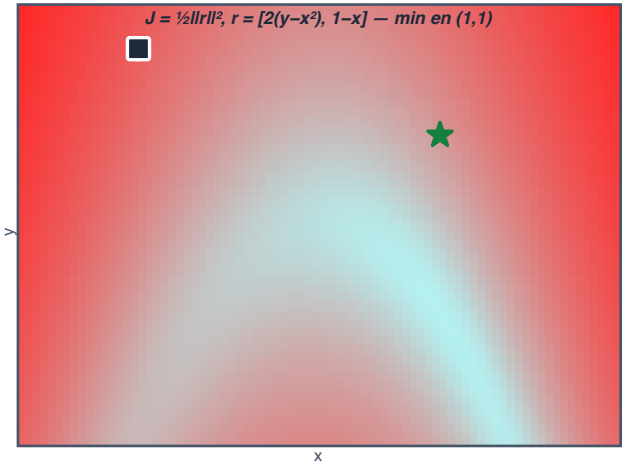
$$H\Delta + \lambda D \Delta = -g$$

$$\boxed{\Delta = -(H + \lambda D)^{-1} g}$$

Both terms matter: we keep the curvature information from GN while damping the step.

A trade-off, **not a weighted average of the two steps**.

LM vs GD vs GN – Trajectories on the Landscape



Comparaison des algorithmes

Gradient Descent (bleu) — Δ = −lr·∇J

J = 3.2500 steps = 0

x=-1.500 y=2.000

Gauss-Newton (rouge) — Δ = −(JᵀJ)⁻¹Jᵀr

J = 3.2500 steps = 0

x=-1.500 y=2.000

Levenberg-Marquardt (vert) — +λI

J = 3.2500 steps = 0

x=-1.500 y=2.000

λ = 1.0000 (λ↑ → GD, λ↓ → GN)

GN : grands pas erratiques loin du min.

LM reste robuste grâce à l'amortissement λ.

x_0 -1.5 y_0 2.0 ▶ Play ↺ Reset étape 0 / 40

Batch vs Incremental

Batch (Global) Optimization:

At each iteration, we solve for Δ^* over **all** variables $X_{0:t}$.

- Accurate but with growing cost.
- In real-time SLAM: the robot operates and accumulates measurements continuously, and the computational cost grows significantly.

Solution: Incremental Inference

Two strategies:

1. **Filtering (EKF)**: keep only the latest pose, marginalize the history.
2. **Windowed smoothing**: keep a sliding window of the last k poses.

Batch vs Incremental

Marginalization ≠ Deletion

Deleting a node = losing the information it contains.

Marginalizing a node = removing it from the graph while **preserving the information** through new factors between its neighbors.

$$p(y) = \int p(x, y)\, dx$$

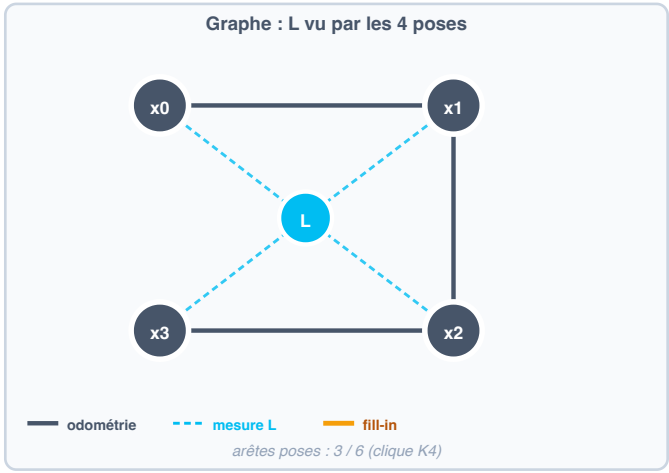
In practice: marginalizing creates new **dense** factors between the remaining variables → the matrix $\mathbf{A}$ becomes less sparse (fill-in).

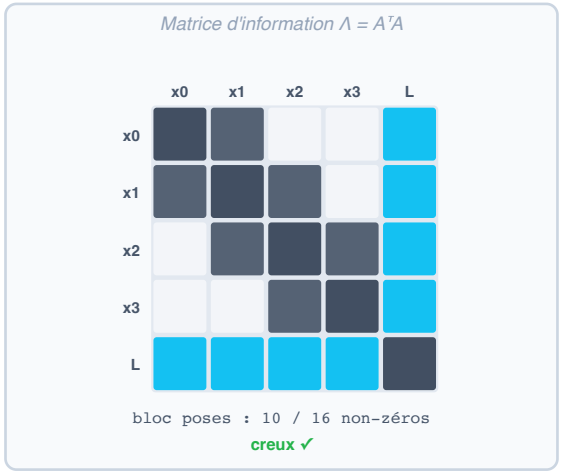
EKF

The Extended Kalman Filter (EKF) marginalizes **all** of the history except the current pose x_t (window=0). It also adds a prediction step.

Allows the robot's position to be estimated well, but does not preserve the map.

Marginalization – Before and After





▶ Marginaliser L ↺ Reset

*Marginaliser ≠ supprimer : retirer L **couple toutes ses voisines** → clique dans le graphe, bloc dense dans Λ (fill-in).*

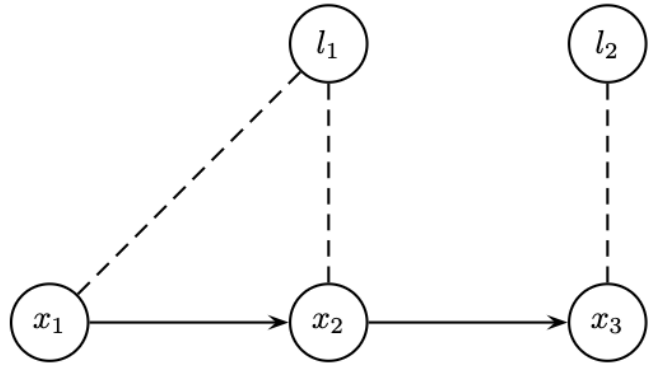
Incremental Inference

In real-time SLAM, information arrives **progressively**: each new step of the robot adds a pose and new measurements.

Key observation: the previous solution Δ^*_{k-1} is an excellent starting point. We must **update** the existing factorization rather than recompute everything.

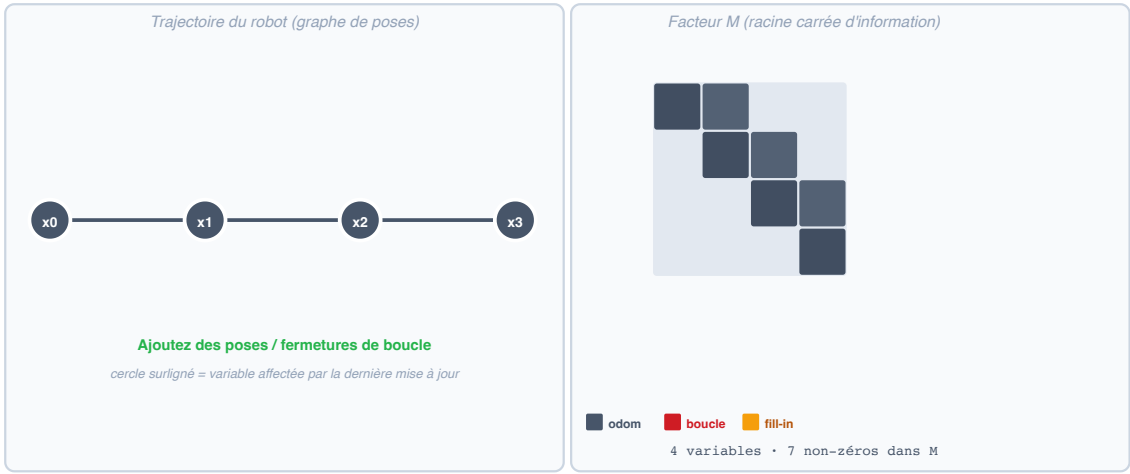
Principle

Adding a measurement = adding a row to the Jacobian $\mathbf{A}$. How can we update $\mathbf{M}$ (the Cholesky factor) efficiently?



Map produced over the long term: the batch problem is hard to maintain at large scale.

Incremental Update



+ Pose (odométrie) **+ Fermeture de boucle** **↺ Reset**

*Incrémental = **local** : odométrie → 1 bloc ; boucle → bande. Jamais de refactorisation complète $O(n^3)$.*

Fixed-Lag Smoothing and Kalman Filtering

Drawback of marginalizing: Does not allow correcting the whole trajectory.

Trade-off: keep a window of L recent poses (lag L) rather than the whole trajectory or only the last pose.

Method	Advantages	Drawbacks
Batch	Accurate and corrects the whole map	costly in computation and memory
Fixed-Lag	Fast	Trajectory (and map) not corrected
EKF	Fast	Does not maintain a corrected map, requires modeling the robot dynamics, may diverge.

Lie Groups

Problem statement – SO(3) is not a Euclidean space

In optimization, the classic update is:

$$X_{k+1} = X_k + \Delta$$

Problem: For rotations $R \in SO(3)$, this addition destroys the properties.

$$R_{\text{new}} = R + \Delta \notin SO(3)$$

because R_{new} no longer satisfies:

- **Orthogonality:** $R^T R = I_3$
- **Right-handedness:** $\det(R) = +1$

Consequence

We cannot use a linear solver directly on rotations. We need a mathematical tool suited to the **geometry** of the space of rotations.

The space SO(3) is a curved manifold

$SO(3)$ is a 3-dimensional **manifold** embedded in $\mathbb{R}^{3\times3}$.

It locally resembles $\mathbb{R}^3$ (we can define a tangent space), but globally it is a curved sphere, not a flat vector space.

Groups and Lie Groups

A **group** G is a set equipped with a binary operation $\otimes$ satisfying:

- **Closure:** $\forall A, B \in G,\ A \otimes B \in G$
- **Associativity:** $(A \otimes B) \otimes C = A \otimes (B \otimes C)$
- **Identity Element:** $\exists I$ such that $A \otimes I = I \otimes A = A$
- **Inverse:** $\forall A,\ \exists A^{-1}$ such that $A \otimes A^{-1} = I$

A **Lie group** is a group whose group operations are differentiable.

Important groups in robotics:

Group	Description	Dim.
$SO(2)$	2D rotations	1
$SO(3)$	3D rotations	3
$SE(2)$	2D poses (R, t)	3
$SE(3)$	3D poses (R, t)	6

Why Lie groups?

They enable a **unified treatment** of rotations and poses, formally define the notion of **distance** between poses, and allow **rigorous optimization** on the manifold (on-manifold optimization).

Distances on SO(3)

Let $R_A, R_B \in SO(3)$.

1. Geodesic Distance (angular)

The minimal angle to align the two frames:

$$d_\theta(R_A, R_B) = \arccos\left(\frac{\mathrm{tr}(R_A^T R_B) - 1}{2}\right)$$

2. Chordal Distance (Frobenius)

Euclidean distance in the space of matrices:

$$d_c(R_A, R_B) = \|R_A - R_B\|_F$$

Analogy on SO(2) – the unit circle:

- The **arc** (path on the circle) = geodesic distance.
- The **chord** (straight line) = chordal distance.

Distances on SE(3)

Let $T_A = (R_A, t_A), T_B = (R_B, t_B) \in SE(3)$.

1. Double Geodesic Distance

Decouples rotation and translation:

$$d_{dg}(T_A, T_B) = \sqrt{d_\theta(R_A, R_B)^2 + \|t_B - t_A\|^2}$$

Problem: incompatible units (radians vs meters).

2. SE(3) Chordal Distance

$$d_c(T_A, T_B) = \|T_A - T_B\|_F$$

$$= \sqrt{d_c(R_A, R_B)^2 + \|t_B - t_A\|^2}$$

Non bi-invariance of SE(3)

Unlike $SO(3)$, distances on $SE(3)$ are not **bi-invariant**:

$$d(T_A, T_B) = d(T_C T_A,\, T_C T_B) \neq d(T_A T_C,\, T_B T_C)$$

This means that the distance between two poses depends on the chosen reference frame. There is no "natural" bi-invariant metric on $SE(3)$.

Lie Algebras: so(3) and se(3)

The Lie algebra is the **tangent space at the identity** of the Lie group.

$\mathfrak{so}(3)$ – Algebra of SO(3)

Set of 3×3 skew-symmetric matrices:

$$\phi^\wedge = [\phi]_\times = \begin{bmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{bmatrix}$$

hat $(\cdot)^\wedge$ (vector → matrix) and **vee** $(\cdot)^\vee$ (matrix → vector) operators.

$\mathfrak{se}(3)$ – Algebra of SE(3)

Perturbation vector $\boldsymbol{\xi} = [\rho^T, \phi^T]^T \in \mathbb{R}^6$:

$$\xi^\wedge = \begin{bmatrix} \phi^\wedge & \rho \\ 0^T & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4}$$

- $\rho \in \mathbb{R}^3$: linear velocity (perturbation of t).
- $\phi \in \mathbb{R}^3$: angular velocity (perturbation of R).

Why the Lie algebra?

The Lie algebra is a **vector space** (flat, linear). This is where we can perform additions and compute gradients, then map back onto the manifold via the exponential map.

Computation on Lie Groups

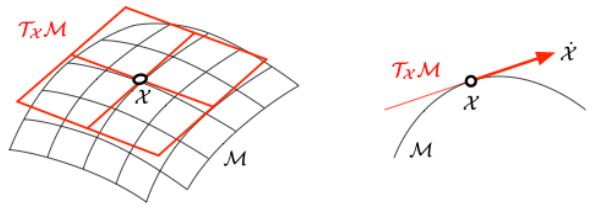
Manifolds and Tangent Space

Definition: A manifold of dimension d is a topological space that **locally** resembles $\mathbb{R}^d$.

Tangent space $T_X\mathcal{M}$: linear vector space tangent to the manifold at the point X.

- Lets us define local **directions of motion**.
- It is the space in which we compute gradients and Jacobians.

A **Lie group** is a smooth manifold whose group operations are differentiable.



The tangent space at a point of the manifold

Intuition

Picture the surface of a sphere. At each point, a tangent plane "sticks" locally to the sphere. We optimize in this plane, then return to the sphere.

Exponential and Logarithmic Maps

These maps connect the **Lie algebra** (tangent space, **linear**) to the **Lie group** (manifold, **nonlinear**) :

Exponential Map : Algebra → Group

$$G = \mathrm{Exp}(\phi) = \exp(\phi^\wedge)$$

For $SO(3)$ – Rodrigues' formula :

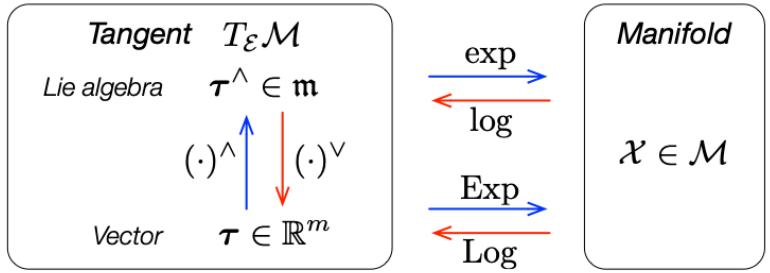
$$R = I + \sin\theta\, u^\wedge + (1 - \cos\theta)\,(u^\wedge)^2$$

with $\phi = \theta u$ (angle $\theta = \|\phi\|$, axis $u = \phi/\|\phi\|$).

Logarithmic Map : Group → Algebra

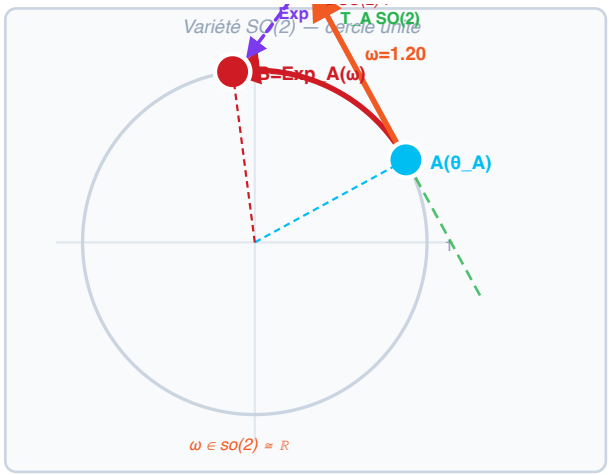
$$\phi = \mathrm{Log}(G) = \log(G)^\vee$$

For $SO(3)$: $\theta = \arccos\left(\frac{\mathrm{tr}(R)-1}{2}\right)$



Summary of the Exp and Log mappings

SO(2) – Manifold, Tangent Space and Exponential Map



Cartes Exp / Log

θ_A = 0.500 rad (28.6°)

θ_B = 1.700 rad (97.4°)

Exp_A(ω) : so(2) → SO(2)

θ_B = θ_A + ω

R(θ_A) =

```
[0.878 -0.479]
[0.479  0.878]
```

R(θ_B) =

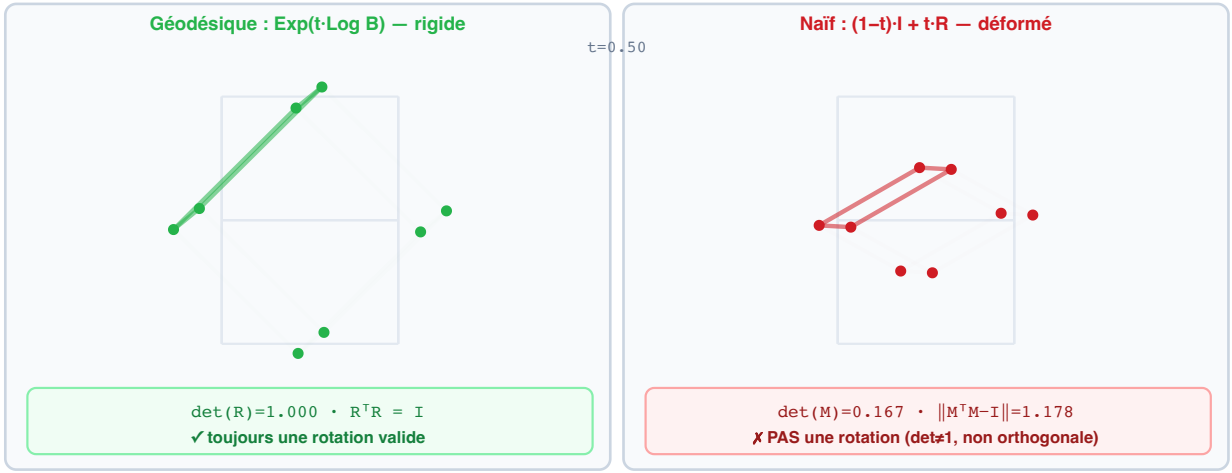
```
[-0.129 -0.992]
[0.992  -0.129]
```

Log_A(B) = ω = θ_B − θ_A = 1.200 ∈ ℝ

θ_A **0.50** ω **1.20**

*L'addition naïve (le long de la tangente) quitte SO(2) · **Exp** rétracte sur la variété → B*

SO(3) – Geodesic vs Naive Addition



t 0.50 ▶ Animer t θ_B 2.30 az 40° el 25°

On ne peut pas additionner des rotations → il faut Exp/Log (géodésique).

Operators ⊕ and ⊖ – Computing on the Manifold

Plus operator $\oplus$: perturb an element of the group :

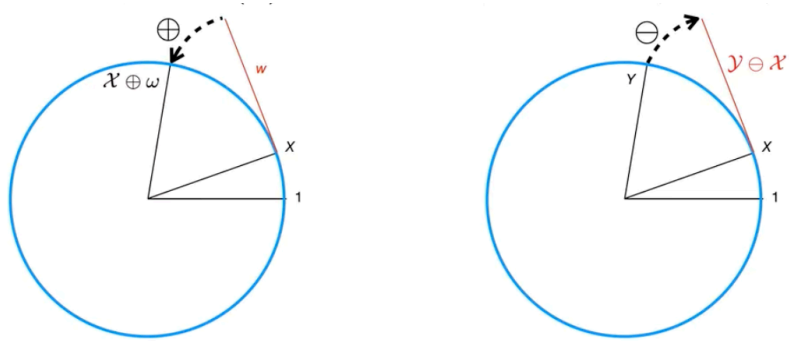
$$\boxed{X \oplus \omega \doteq X \cdot \mathrm{Exp}(\omega)}$$

- $X \in G$: current state (on the manifold).
- $\omega \in \mathbb{R}^n$: perturbation in the tangent space (vector).
- $X \oplus \omega \in G$: new state (on the manifold).

Minus operator $\ominus$: difference between elements :

$$\boxed{X \ominus Y \doteq \mathrm{Log}(Y^{-1} \cdot X)}$$

Gives the perturbation in the tangent space that transforms Y into X.



Operators ⊕ and ⊖ on the manifold

Lift – Solve – Retract

The optimization paradigm on manifolds, used with a solver such as Gauss-Newton and L-M.

① Lift

We **transfer** the problem from the curved manifold to the local **linear tangent space** at X_k.

We compute the residuals $r_i = h_i(X_k) - z_i$ and the Jacobians $J_i = \frac{\partial h_i}{\partial X}\big|_{X_k}$.

② Solve

In the tangent space (**vector, flat**), we compute the optimal step via the normal equations:

$$(J^T J)\,\Delta^* = -J^T r$$

Solved by Cholesky, exploits the **sparsity** of the factor graph.

③ Retract

We **bring** the increment Δ^* back onto the manifold via the $\oplus$ operator:

$$X_{k+1} = X_k \oplus \Delta^*$$

The resulting X_{k+1} is a valid element of the group (rotation, pose…).

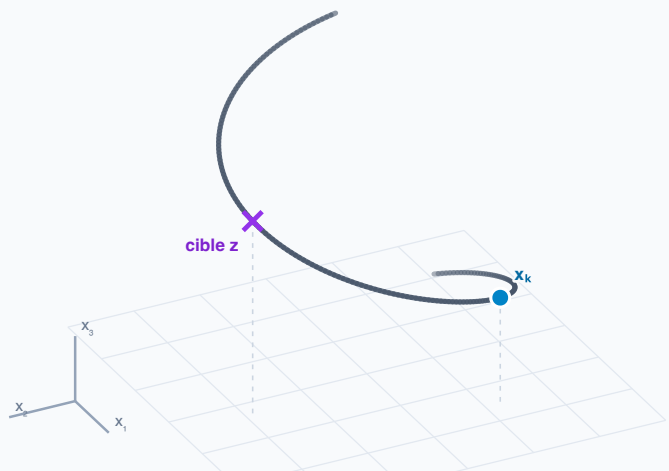
Iterations until convergence

Lift — Solve — Retract on a 3D Curve

Le problème | ① **Lift · linéariser** | ② **Solve · calculer** | ③ **Retract · revenir**

Une courbe 1D dans $\mathbb{R}^3$

Glisser pour tourner · touches fléchées



— Hélice ● État courant ○ Hors courbe ● Rétraction

Un état contraint à rester sur l'hélice

$$q(t) = \begin{bmatrix} \cos t \\ \sin t \\ 0.35t \end{bmatrix}, \quad x_k = q(t_k)$$

La position x a **3 coordonnées**, mais un seul degré de liberté : le paramètre **t**.

$$\min_t F(t) = \tfrac{1}{2}\|q(t) - z\|^2$$

On cherche à rejoindre la cible violette **z = q(1,35)**, située sur la courbe.

> Une droite tangente suffit : la dimension de l'espace tangent est **1, pas 3**.